This would look for a partial named `_link_area.html.erb` and render it using the layout `_graybar.html.erb`. Note that layouts for partials follow the same leading-underscore naming as regular partials, and are placed in the same folder with the partial that they belong to (not in the master `layouts` folder).
Also note that explicitly specifying `:partial` is required when passing additional options such as `:layout`.

3.4.4 Passing Local Variables

You can also pass local variables into partials, making them even more powerful and flexible. For example, you can use this technique to reduce duplication between new and edit pages, while still keeping a bit of distinct content:

- new.html.erb
```
<h1>New zone</h1>
<%= render partial: "form", locals: {zone: @zone} %>
```
- edit.html.erb
```
<h1>Editing zone</h1>
<%= render partial: "form", locals: {zone: @zone} %>
```
- _form.html.erb
```
<%= form_for(zone) do |f| %>
  <p>
    <b>Zone name</b><br>
    <%= f.text_field :name %>
  </p>
  <p>
    <%= f.submit %>
  </p>
<% end %>
```

Although the same partial will be rendered into both views, Action View's submit helper will return "Create Zone" for the new action and "Update Zone" for the edit action.

To pass a local variable to a partial in only specific cases use the `local_assigns`.

- index.html.erb
```
<%= render user.articles %>
```
- show.html.erb
```
<%= render article, full: true %>
```
- _articles.html.erb
```
<%= content_tag_for :article, article do |article| %>
  <h2><%= article.title %></h2>

  <% if local_assigns[:full] %>
    <%= simple_format article.body %>
  <% else %>
    <%= truncate article.body %>
  <% end %>
<% end %>
```

This way it is possible to use the partial without the need to declare all local variables.

Every partial also has a local variable with the same name as the partial (minus the underscore). You can pass an object in to this local variable via the `:object` option:

```
<%= render partial: "customer", object: @new_customer %>
```

Within the `customer` partial, the `customer` variable will refer to `@new_customer` from the parent view.
If you have an instance of a model to render into a partial, you can use a shorthand syntax:

```
<%= render @customer %>
```